

BSIT MELEC 9: System Need Analysis

Module 2 Study Reviewer & Practice Examination

COURSE: BSIT - MELEC 9 (System Need Analysis)	REFERENCE: BSIT_LS_MELEC9_N02.docx (Learning Sheet No. 2)
TOPIC: Software Development Lifecycle (SDLC)	COVERAGE: Prelim Examination
Key Competencies: Master the 7 phases of SDLC, key deliverables, testing methodologies (Unit, Integration, System, UAT), and lifecycle pros/cons.	Format: Notes + Practice Test + Answer Key

I. Definition and Core Purpose of the SDLC

The **Software Development Life Cycle (SDLC)** is a structured, step-by-step methodology used by software teams to plan, design, build, test, deploy, and maintain information systems throughout their entire operational life.

Why is SDLC Indispensable?

Without a disciplined lifecycle, software development devolves into chaos—deadlines are missed, costs skyrocket, code is buggy, and the final application fails to solve the client's actual problems. SDLC provides a predictable roadmap that guarantees quality, accountability, and cost control.

II. The Seven (7) Sequential Phases of the SDLC

In the foundational linear/waterfall representation, each phase produces specific tangible deliverables that serve as prerequisites for the succeeding stage:

Phase No. & Name	Primary Objective & Activities	Key Deliverable / Output
1. Planning	Determine project feasibility, estimate budget, establish timelines, identify risks, and assign team personnel.	Project Charter, Feasibility Study, Budget & Schedule
2. Requirements Analysis	Gather detailed business needs from stakeholders via interviews, surveys, and observation.	Software Requirements Specification (SRS)
3. System Design	Architect the technical blueprint: database ERDs, UI wireframes, process flowcharts, and security rules.	System Design Document (SDD), UI Mockups, DB Schema
4. Development (Coding)	Programmers write executable source code, construct databases, and integrate frontend and backend APIs.	Working Source Code / Functional Software
5. Testing	Execute test cases to detect and remove logic errors, performance flaws, security vulnerabilities, and bugs.	Test Cases, Bug Reports, Validated Software
6. Deployment	Install software on production servers, migrate data, train end-users, and conduct formal launch (go-live).	Operational / Live Production System
7. Maintenance	Ongoing support, patching security vulnerabilities, fixing newly discovered bugs, and adding new features.	Software Patches, Maintenance Logs, Version Updates

EXAM TIP / CRITICAL CONCEPT: SDLC SEQUENTIAL FLOW

Exam questions love to ask: *'What phase immediately precedes Coding?'* (System Design) or *'In what phase is user training conducted?'* (Deployment). Memorize the strict order: **Planning → Requirements Analysis → System Design → Development → Testing → Deployment → Maintenance.**

III. Deep-Dive: Phase 5 Software Testing Methodologies

Software testing is the systematic verification that a program behaves exactly as designed and satisfies customer requirements without unexpected failures. There are four (4) progressive levels of testing:

Testing Level	Focus & Scope	Real-World Enrollment System Example
1. Unit Testing	Verifying individual functions, methods, or isolated code components independently.	Testing if the computeTuitionFee() function returns exact amounts when given 18 credit units.
2. Integration Testing	Verifying the interfaces and data exchange between two or more integrated modules.	Testing if the Student Registration module successfully passes student IDs to the Payment Module.

3. System Testing	Verifying the complete, fully integrated application as a whole against the entire SRS.	Testing complete end-to-end performance, security, and load with 2,000 concurrent students.
4. User Acceptance Testing (UAT)	Formal testing performed by real end-users (clients) in operational conditions to decide acceptance.	The University Registrar and Cashier test the system to formally certify that it meets institutional rules.

IV. Advantages and Disadvantages of the SDLC

Advantages of Utilizing SDLC	Challenges / Disadvantages of Traditional SDLC
• Ensures high software quality and operational reliability.	• Can take significant time and cause perceived project delays.
• Drastically reduces coding defects and logic errors.	• Requires exhaustive upfront documentation and paperwork.
• Improves resource management and prevents budget overruns.	• Changing client requirements mid-way can trigger high rework costs.
• Guarantees full alignment with documented user needs.	• Highly rigid sequential models struggle with rapidly changing markets.
• Makes post-deployment maintenance organized and simple.	• Requires experienced project managers and disciplined developers.

MODULE 2: SAMPLE EXAMINATION

Test your mastery of Module 2. Complete all parts before checking the Answer Key.

PART I: MULTIPLE CHOICE (10 Items)

Select the best answer for each question.

1. What is the official tangible deliverable produced at the conclusion of the Requirements Analysis phase?
 - A. Entity-Relationship Diagram (ERD)
 - B. Software Requirements Specification (SRS)
 - C. Executable Application File (.exe)
 - D. Post-Implementation Review Report
2. In which phase of the SDLC do programmers physically write application source code and configure databases?
 - A. System Design
 - B. Development (Coding)
 - C. Requirements Analysis
 - D. Deployment
3. Testing an individual function in isolation (such as a discount calculation formula) is classified as:
 - A. User Acceptance Testing (UAT)
 - B. Integration Testing
 - C. Unit Testing
 - D. Stress Testing
4. Which SDLC phase occurs immediately after System Design is officially approved?
 - A. Testing
 - B. Planning
 - C. Development (Coding)
 - D. Deployment
5. Conducting user training, migrating legacy databases to production servers, and officially launching the software belongs to:
 - A. Maintenance
 - B. Deployment
 - C. System Design
 - D. Feasibility Analysis
6. A university registrar officially validates that the enrollment system operates according to university policies before sign-off. What type of testing is this?
 - A. Unit Testing
 - B. Regression Testing
 - C. User Acceptance Testing (UAT)
 - D. Syntax Checking
7. Six months after deployment, a university decides to add an automated GCash payment gateway. Which phase manages this modification?
 - A. Maintenance
 - B. System Design
 - C. Planning
 - D. Unit Testing
8. Creating UI wireframes, page mockups, and database table structures occurs during which phase?
 - A. System Design
 - B. Requirements Analysis
 - C. Development
 - D. Maintenance
9. Which phase is widely recognized as the foundational phase determining project viability and budget constraints?
 - A. Planning

- B. Coding
- C. Testing
- D. Deployment

10. Testing whether the Student Registration module correctly transfers student data into the Billing module is an example of:

- A. Unit Testing
- B. Integration Testing
- C. Acceptance Testing
- D. Alpha Testing

PART II: IDENTIFICATION / SEQUENCE (5 Items)

Identify the term or phase being described.

- 11. The type of testing conducted to ensure that distinct software modules interface and exchange data seamlessly: _____
- 12. The final phase of the SDLC where software is continuously patched, enhanced, and supported post-launch: _____
- 13. The deliverable produced during the System Design phase that outlines database tables and relationships: _____
- 14. The phase that answers the core question: 'What should the software actually do from the user's perspective?': _____
- 15. The phase during which end-users undergo formal training and the software is installed onto operational servers: _____

PART III: TRUE OR FALSE (5 Items)

Write *TRUE* if the statement is correct; write *FALSE* if the statement is incorrect.

- 16. In standard SDLC methodology, programmers should begin writing code prior to finalizing system requirements and design blueprints.
- 17. User Acceptance Testing (UAT) is performed exclusively by internal software developers without involving any actual end-users.
- 18. The Software Requirements Specification (SRS) acts as an agreed-upon contract between the client and the development team.
- 19. Maintenance is often the longest and most resource-intensive phase across the full operational life of enterprise software.
- 20. One recognized drawback of the traditional SDLC is that changing requirements mid-development can significantly escalate costs.

PART IV: SCENARIO APPLICATION & ESSAY (2 Items)

Provide comprehensive, analytical answers for each question.

Question 21: Imagine you are assigned to lead the development of an **Online Library Management System** for your college. List at least **five (5) specific user requirements** that must be gathered and documented during the *Requirements Analysis* phase.

Question 22: Why is comprehensive software testing indispensable prior to live deployment? Identify and describe at least **three (3) catastrophic real-world consequences** that testing prevents in an enterprise system.

MODULE 2: ANSWER KEY & DETAILED RATIONALES

Question 1. B (Software Requirements Specification - SRS)

Rationale: The SRS is the formal document containing all functional and non-functional requirements compiled during analysis.

Question 2. B (Development / Coding)

Rationale: Actual programming and database schema creation occur exclusively during Phase 4 (Development).

Question 3. C (Unit Testing)

Rationale: Unit testing tests individual subroutines, modules, or isolated functions in complete isolation.

Question 4. C (Development / Coding)

Rationale: Once the blueprint (Design) is reviewed and approved, developers proceed to write the program code.

Question 5. B (Deployment)

Rationale: Deployment involves server setup, database seeding, user training, and launching the system into operational use.

Question 6. C (User Acceptance Testing - UAT)

Rationale: UAT is the final gate where actual end-users confirm the system solves their daily business needs before accepting delivery.

Question 7. A (Maintenance)

Rationale: Adding enhancements, integrating new APIs, and updating software post-deployment is the primary mandate of Maintenance.

Question 8. A (System Design)

Rationale: System Design creates the structural layout: mockups, schemas, architecture diagrams, and interface layouts.

Question 9. A (Planning)

Rationale: Planning establishes project feasibility, timelines, team structure, and financial budgets.

Question 10. B (Integration Testing)

Rationale: Integration testing specifically verifies that data flows accurately across module boundaries without corruption.

PART II: IDENTIFICATION ANSWERS

- 11. Integration Testing — Testing data exchange and communication between combined software units.
- 12. Maintenance — The ongoing phase of sustaining, updating, and patching deployed software.
- 13. Entity-Relationship Diagram (ERD) / Database Schema — The visual model specifying entities, attributes, and relationships.
- 14. Requirements Analysis — Phase focused on extracting and documenting user and business requirements.
- 15. Deployment — Phase encompassing server rollout, configuration, and end-user onboarding.

PART III: TRUE OR FALSE ANSWERS

- 16. FALSE — Writing code before establishing clear requirements and architecture causes immense rework and design failure.
- 17. FALSE — UAT must be conducted by actual clients/end-users; developers conduct unit, integration, and system tests.
- 18. TRUE — The SRS is a binding baseline specification agreed upon by both stakeholders and technical teams.
- 19. TRUE — Maintenance typically lasts for years throughout the operational lifespan of the software.
- 20. TRUE — In sequential models, changing requirements midway requires backtracking through design and coding, which inflates cost.

PART IV: SCENARIO APPLICATION SAMPLE ANSWERS

Model Answer for Question 21 (Online Library System Requirements):

- 1. *Book Search & Cataloging:* Students can search books by title, author, ISBN, or genre and view real-time availability.
- 2. *Automated Borrowing & Reservation:* Registered students can reserve a book online for pickup within 24 hours.
- 3. *Penalty & Fine Computation:* The system must automatically calculate daily overdue fines (e.g. Php 10/day) upon late return.
- 4. *Account & Circulation History:* Users can view their active borrowed books, due dates, and past transaction history.
- 5. *Librarian Inventory Management:* Staff can add new book acquisitions, archive damaged copies, and generate overdue reports.

Model Answer for Question 22 (Why Software Testing is Crucial):

Software testing is critical because deploying unverified code into production risks operational collapse, financial loss, and severe legal liability.

Three Consequences Prevented by Testing:

- 1. *Data Corruption & Inaccurate Calculations:* E.g., computing negative tuition or dropping enrolled students due to uncaught edge-case bugs.
- 2. *System Crashes & High-Traffic Downtime:* Testing ensures servers handle peak traffic (e.g., thousands of simultaneous students during enrollment day) without crashing.
- 3. *Security Breaches & Data Leaks:* Testing uncovers SQL injection and broken authentication vulnerabilities, safeguarding sensitive student records from hackers.